Deep Learning for Structural Design

Lecture 2 – Topology optimization, generative models, inverse design

Dr. L. Lomazzi

April 21, 2026

Roadmap

1. **Part A** – The design problem under dynamic loads
2. **Part B** – Topology optimization meets deep learning
3. **Part C** – Generative models for design (AE, VAE, GAN)
4. **Part D** – Inverse design
5. **Part E** – Synthesis and handoff

From Lecture 1 to Lecture 2

Lecture 1 built surrogates that *predict* the response of a given design. Now we use them – together with generative models – to *propose* designs.

L2 – Forward vs. inverse problems

Forward problem

- Given design d, load p.
- Predict performance y via FE or surrogate.
- *One-to-one, well posed.*

Inverse problem

- Given target performance $y^\star$.
- Find design(s) d that achieve it.
- *Often many solutions – ill-posed.*

Two big moves this lecture

1. Use DL **inside** classical design loops (topology optimization).
2. Use DL to **replace** the loop (generative / inverse design).

L2 – Design under dynamic loads is harder

Compared to static design:

- **Nonlinear response.** Plasticity, damage, contact.
- **Time-dependent objectives.** Peak acceleration, absorbed energy, residual deflection.
- **Rate-dependent materials.** Properties depend on strain rate.
- **Multi-objective.** Light *and* robust *and* manufacturable.

Why classical optimization struggles

Every objective evaluation is an **explicit-dynamics FE run**. Gradient-based loops need sensitivities that are expensive or unavailable.

L2 – Where DL helps the design pipeline

Three complementary roles:

- **Surrogate in the loop.** Replace FE evaluations with a fast surrogate (from Lecture 1).
- **Learned design generator.** Map boundary conditions $\rightarrow$ optimized topology directly.
- **Learned design prior.** Generative models produce feasible, diverse candidates.

Common thread

All three *amortize* the cost of many FE runs into one training phase.

L2 – SIMP: the design variable

Given a design domain Ω, loads F, and supports, where should material be *placed* to make the structure as stiff as possible under a mass budget?

- Partition Ω into N_e finite elements $e = 1, \ldots, N_e$.
- For each element, introduce a **density variable** $\rho_e \in [0, 1]$:
  - $\rho_e = 1$ – element is solid material.
  - $\rho_e = 0$ – element is void (no material).
  - $\rho_e \in (0, 1)$ – *relaxed* intermediate value, so we can use gradients.

Interpolation (the SIMP rule). The element stiffness matrix scales with density as

$$\mathrm{K}_e(\rho_e) \;=\; \rho_e^p \, \mathrm{K}_e^0, \qquad p \approx 3,$$

where K_e^0 is the fully-solid element stiffness matrix.

Why the exponent p?

$p > 1$ penalizes intermediate densities. This pushes the optimizer toward a **black-and-white** (solid / void) design rather than a gray continuum.

L2 – SIMP: the optimization problem

Minimize compliance (= maximize stiffness) under a volume budget:

$$\min_{\boldsymbol{\rho}} \quad c(\boldsymbol{\rho}) \;=\; \mathsf{U}^\top \mathsf{K}(\boldsymbol{\rho})\,\mathsf{U}$$

$$\text{s.t.} \quad \underbrace{\mathsf{K}(\boldsymbol{\rho})\,\mathsf{U} \;=\; \mathsf{F}}_{\text{FE equilibrium}}, \qquad \underbrace{\frac{1}{V_0}\sum_{e=1}^{N_e} v_e\,\rho_e \;\le\; f}_{\text{volume} \le \text{fraction } f \text{ of } V_0}, \qquad \rho_e \in [0,1].$$

Variables.

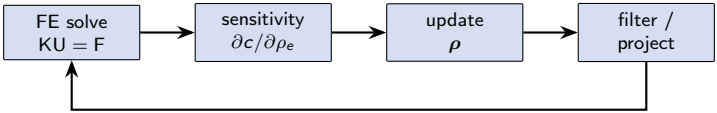
- $\boldsymbol{\rho} = (\rho_1, \ldots, \rho_{N_e})$ – vector of element densities (the design).
- $\mathsf{K}(\boldsymbol{\rho}) = \sum_e \rho_e^p\,\mathsf{K}_e^0$ – global stiffness matrix (assembled from elements).
- U – nodal displacements, F – applied load vector.
- $c(\boldsymbol{\rho}) = \mathsf{U}^\top \mathsf{K}\mathsf{U}$ – **compliance**; low $c \Leftrightarrow$ stiff structure.
- v_e – volume of element e; $V_0 = \sum_e v_e$; $f \in (0,1)$ – target volume fraction (e.g. $f = 0.4$).

L2 – SIMP: sensitivity and the iteration loop

To update $\boldsymbol{\rho}$ with a gradient method, we need $\partial c/\partial \rho_e$. Properties worth noting:

- $\partial c/\partial \rho_e$ *is always negative* – adding material ($\rho_e \uparrow$) always reduces compliance.
- *Cheap* – once U is known, sensitivities for all elements are just element-wise dot products.

The SIMP iteration loop.



Where the cost lives

One **FE solve per iteration**, $\sim$ 100–1000 iterations per design. This is the bottleneck DL can attack.

L2 – Where does it become painful?

- **One FE solve per iteration**, often 100–1000 iterations per design.
- **Scales badly with resolution.** 3D, fine meshes, dynamic loads $\Rightarrow$ solve cost dominates.
- **Sensitivities** may be unavailable for nonlinear / dynamic / contact problems.
- **Reruns** for every new boundary condition, load case, or material.

Opportunity for DL

Everything expensive (FE solve, sensitivity, whole pipeline) is a candidate for amortization.

L – Three flavors of DL-accelerated TO

(a) Learn the optimizer step.

- Input: current density $\boldsymbol{\rho}^{(k)}$, BCs. Output: update $\Delta\boldsymbol{\rho}^{(k)}$.
- Keeps FE-in-the-loop but removes repeated sensitivity work.

(b) Direct design predictor (one-shot TO).

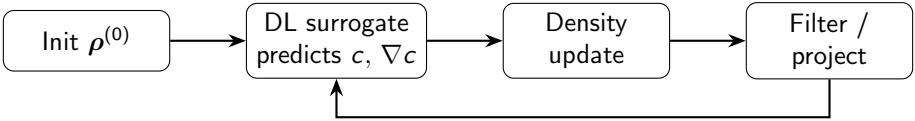
- Input: BCs + load + volume fraction. Output: final optimized topology.
- Huge inference speedup; generalization outside training BCs is the key question.

(c) Neural reparameterization.

- Instead of optimizing the density vector $\boldsymbol{\rho}$ directly, represent it as the output of a neural network: $\rho_e = g_\theta(\mathrm{x}_e)$, where x_e is the element coordinate.
- **Optimize** θ, not $\boldsymbol{\rho}$, on the *same* compliance objective – no training data needed.
- The NN's smoothness acts as a built-in filter against checkerboard artifacts.

L2 – Flavor (a) in detail: surrogate inside the TO loop

The most common implementation of flavor (a): replace the FE solve with a learned surrogate, keep the rest of the SIMP loop unchanged.



- The surrogate (ideally) provides **differentiable sensitivities** directly via autograd.
- **Active learning.** Periodically query the true FE solver on new designs and retrain, so the surrogate stays accurate as the optimizer explores new regions.

L2 – The three flavors side by side

	(a) Surrogate in loop	**(b) Direct predictor**	**(c) Neural reparam.**
What the NN does	predicts c, ∇c from $\boldsymbol{\rho}$	maps BCs $\rightarrow$ topology	parameterizes $\rho_e = g_\theta(\mathrm{x}_e)$
Loop structure	classical SIMP	no loop (one-shot)	classical SIMP
Replaces	FE solve	whole pipeline	density vector $\boldsymbol{\rho}$
Training data	(design, c) pairs	(BC, topology) pairs	none
Cost per new BC	FE + surrogate retraining	$\sim$ zero (just inference)	full re-optimization

Observation

(a) and (b) are **data-driven**: they need a dataset of previously optimized designs.
(c) is **data-free**: the NN is just a parameterization, driven by the compliance objective directly.

L2 – Limits of DL-accelerated TO

- **Generalization across BCs.** A model trained on one loading family may fail when loads rotate or scale.
- **Distribution shift.** Optimized designs often live at the edges of the training distribution.
- **Manufacturability.** DL can produce checkerboards, disconnected features, unprintable overhangs.
- **Trust.** If the surrogate is wrong, the final topology is wrong. V&V is mandatory.

Practical stance

Treat DL-accelerated TO as a **screening tool**: generate candidates fast, then validate the best ones with full FE.

L2 – Why generative?

TO finds *a* good design for one problem. Generative models go further:

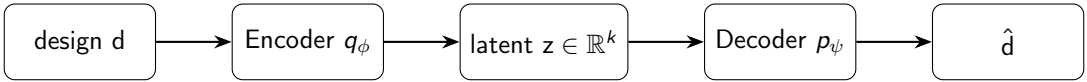
- **Diverse candidates.** Many distinct designs with similar performance.
- **Novelty.** Designs outside human intuition.
- **Conditioning.** Generate designs meeting performance targets directly.
- **Exploration.** Smoothly interpolate between existing designs.

High-level idea

Learn a probability distribution $p(\mathrm{d})$ over *feasible, high-performing designs* – then sample from it.

L2 – Autoencoder: the building block

An autoencoder (AE) learns a compressed representation of a design.



Training. Minimize reconstruction loss $\|d - \hat{d}\|^2$ on a dataset of designs.

Limitation of a plain AE

The latent space is **not structured**: a random z rarely decodes to a meaningful design.

L2 – Variational Autoencoder (VAE): the probabilistic picture

Treat the latent z as a *random variable* with prior $p(\mathrm{z}) = \mathcal{N}(0, \mathrm{I})$.

- **Encoder** $q_\phi(\mathrm{z} \mid \mathrm{d})$: outputs a distribution, parameterized as $\mathcal{N}\Big(\boldsymbol{\mu}_\phi(\mathrm{d}), \boldsymbol{\sigma}^2_\phi(\mathrm{d})\Big)$.
- **Decoder** $p_\psi(\mathrm{d} \mid \mathrm{z})$: (usually) deterministic map back to the design space.

Reparameterization trick. Sample $\mathrm{z} = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon},\ \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathrm{I})$ – makes the sampling step differentiable.

What we gain

A latent space with **structure**: nearby codes $\rightarrow$ similar designs, and sampling from $\mathcal{N}(0, \mathrm{I})$ produces plausible new ones.

L2 – VAE loss: the ELBO

Maximize the Evidence Lower BOund:

$$\mathcal{L}_{\text{VAE}}(\phi, \psi) = \underbrace{\mathbb{E}_{q_\phi(\mathsf{z}|\mathsf{d})}[\log p_\psi(\mathsf{d} \mid \mathsf{z})]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}(q_\phi(\mathsf{z} \mid \mathsf{d}) \,\|\, p(\mathsf{z}))}_{\text{regularizer}}$$

- **Reconstruction term** – standard loss (MSE for fields, BCE for binary masks).
- **KL term** – pushes the encoder distribution towards the prior $\Rightarrow$ keeps the latent space continuous and compact.

Intuition

The KL term is what makes a VAE *generative*: without it, we just have an autoencoder.

L2 – Minimal VAE skeleton

```python
class VAE(nn.Module):
    def __init__(self, d_in, k):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(d_in, 128), nn.ReLU(),
                                 nn.Linear(128, 2*k)) # mu and log-var
        self.dec = nn.Sequential(nn.Linear(k, 128), nn.ReLU(),
                                 nn.Linear(128, d_in))
    def forward(self, x):
        mu, logv = self.enc(x).chunk(2, dim=-1)
        z = mu + torch.exp(0.5*logv) * torch.randn_like(mu) # reparam.
        return self.dec(z), mu, logv

def vae_loss(x_hat, x, mu, logv):
    rec = ((x_hat - x)**2).sum(dim=-1).mean()
    kld = -0.5*(1 + logv - mu.pow(2) - logv.exp()).sum(dim=-1).mean()
    return rec + kld
```

L2 – VAE for design: a latent space of shapes

- **Training data**: a library of existing / optimized designs (binary masks, density fields, CAD).
- **Learned latent**: a smooth, low-dimensional parameterization of "plausible designs".
- **Sampling** $\mathsf{z} \sim \mathcal{N}(0, \mathsf{I}) \Rightarrow$ new candidate designs.
- **Interpolation** between two known designs $\mathsf{d}_1, \mathsf{d}_2$: decode $\alpha \mathsf{z}_1 + (1-\alpha)\mathsf{z}_2$.

Engineering payoff

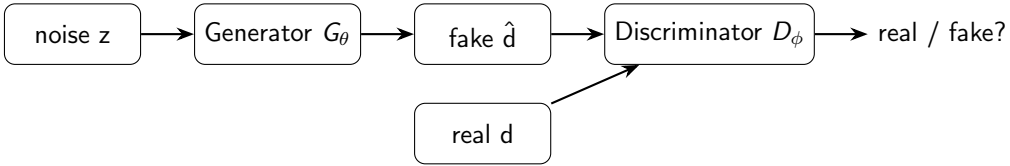
The latent space reduces the design search from millions of voxels to a handful of meaningful coordinates – ideal for inverse design.

L2 – Generative Adversarial Networks (GANs)

Two networks in a game:

- **Generator** G_θ: maps noise $\mathrm{z} \sim \mathcal{N}(0, \mathrm{I})$ to a candidate design.
- **Discriminator** D_ϕ: distinguishes real designs from generated ones.

They train by competing: G tries to fool D; D tries to classify correctly.



L2 – GAN objective and training dynamics

$$\min_{\theta} \max_{\phi} \; \mathbb{E}_{\mathrm{d}\sim p_{\mathrm{data}}}[\log D_\phi(\mathrm{d})] + \mathbb{E}_{\mathrm{z}\sim p(\mathrm{z})}[\log(1 - D_\phi(G_\theta(\mathrm{z})))]$$

Intuition.

- Discriminator tries to push $D(\mathrm{d}) \to 1$ on real, $D(G(\mathrm{z})) \to 0$ on fakes.
- Generator tries to push $D(G(\mathrm{z})) \to 1$.
- Alternating updates $\Rightarrow$ G progressively matches p_{data}.

Known failure modes

Mode collapse: G produces only a few design types.
Training instability: oscillations, vanishing gradients.
Modern variants (WGAN, StyleGAN) mitigate but do not fully eliminate these.

L2 – VAE vs. GAN for design

	VAE	**GAN**
Samples	slightly blurry	sharp, high-fidelity
Latent space	smooth, structured, usable	less interpretable
Training	stable	tricky, unstable
Diversity	broad coverage	risk of mode collapse
Inverse design	easy via latent optimization	harder

Practical guideline

Start with a VAE to understand the design space; move to a GAN if output sharpness is critical (e.g. fine lattice geometry).

L2 – Conditional generation

Plain generative models sample from $p(\mathrm{d})$. For design we want $p(\mathrm{d} \mid \mathrm{c})$, where c encodes constraints:

- target performance (e.g. absorbed energy $\geq E^\star$),
- loading case (impulse magnitude, angle),
- material / manufacturing process.

How. Feed c as an extra input to both encoder and decoder (CVAE) or generator and discriminator (cGAN).

Why this is the engineering target

Unconditional sampling is a curiosity. **Conditional generation** is what turns a generative model into an inverse-design tool.

L2 – Diffusion models: the two processes

Learn to *reverse* a gradual noising process. Generating a new design = starting from pure noise and denoising it, step by step.

Forward process (fixed, no learning). Take a real design d_0 from the training set and corrupt it in T small steps (typically $T \sim 1000$):

$$\mathrm{d}_t \;=\; \sqrt{1-\beta_t}\,\mathrm{d}_{t-1} \;+\; \sqrt{\beta_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathrm{I}),$$

with a small noise schedule $\beta_t \in (0, 1)$. After T steps, d_T is essentially pure Gaussian noise.

Reverse process (what the NN learns). A network $\boldsymbol{\epsilon}_\theta(\mathrm{d}_t, t)$ predicts the noise that was added, so we can "undo" one step:

$$\mathrm{d}_{t-1} \;\approx\; \mathrm{denoise}\big(\mathrm{d}_t,\; \boldsymbol{\epsilon}_\theta(\mathrm{d}_t, t)\big).$$

L2 – Diffusion models: the training loss and sampling

Training loss. Pick a random step t, add the corresponding noise to a training design, and train the network to predict the noise:

$$\mathcal{L}(\theta) \;=\; \mathbb{E}_{\mathsf{d}_0,\, t,\, \boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon} \;-\; \boldsymbol{\epsilon}_\theta(\mathsf{d}_t, t) \right\|^2.$$

Surprisingly simple: plain MSE between true and predicted noise.

Sampling a new design.

Iterate the reverse step T times. Each call uses the *same* trained network $\boldsymbol{\epsilon}_\theta$.

L2 — Diffusion models: conditioning, practice

Conditional generation. To target a performance $y^\star$ (e.g. absorbed energy), feed $y^\star$ as an extra input:

$$\epsilon_\theta(d_t,\, t,\, y^\star) \quad \Rightarrow \quad \text{designs conditioned on } y^\star.$$

Same loop, different inputs. This is what makes diffusion attractive for inverse design.

Trade-offs vs. VAE / GAN.

- + Stable training, high diversity, no mode collapse, strong conditioning.
- + State of the art for image and 3D-shape generation today.
- – Inference is T network calls ($\sim$ 50–1000), vs. 1 for VAE / GAN. Actively being reduced (distillation, latent diffusion, consistency models).

Engineering uptake

Early but growing: conditional diffusion for topology optimization, inverse design of metamaterials and airfoils.

L2 – Evaluating generated designs

Good reconstruction $\neq$ good engineering design. **Three-axis check:**

- **Validity.** Connected? Boundary-satisfying? Manufacturable?
- **Performance.** Simulate (FE or surrogate) and measure the actual QoI.
- **Novelty / diversity.** Distance in latent space, coverage of design space.

Rule

A generative model that produces beautiful but invalid or identical designs is worse than useless.

L2 – Inverse design, stated cleanly

Given a target performance $y^\star$, find a design d with $F(d) \approx y^\star$ and constraints satisfied.

Difficulties.

- F is expensive (FE solver) – so we use DL surrogate models.
- The problem is often **non-unique**: many designs hit the same target.
- The problem is often **ill-conditioned**: small changes in $y^\star \to$ large changes in d.

Three strategies

A. Gradient through a surrogate.
B. Latent-space search.
C. Direct conditional generation.

L2 – Strategy A: gradient through a differentiable surrogate

Train a surrogate $f_\theta : \mathrm{d} \to \mathrm{y}$, then:

$$\min_{\mathrm{d}} \ \|f_\theta(\mathrm{d}) - \mathrm{y}^\star\|^2 \ + \ \lambda\,\mathcal{R}(\mathrm{d}) \qquad \text{via gradient descent on d.}$$

- $\mathcal{R}$: regularizer enforcing feasibility (volume, smoothness, symmetry).
- Gradients via autograd through f_θ.

Strengths / weaknesses

+ Simple to implement; leverages any surrogate.
– Optimization may leave the training distribution; the surrogate then extrapolates.

L2 – Strategy B: latent-space optimization

Use a trained VAE (or GAN): designs live on a low-dim latent manifold.

$$\min_{z} \quad \|f_\theta(\mathrm{Dec}(z)) - y^\star\|^2 \qquad d = \mathrm{Dec}(z).$$

- Optimization happens in z, not d: few dimensions, smooth landscape.
- The decoder acts as a **learned feasibility prior**: staying on the manifold keeps designs plausible.

Best used when

You have a reasonable dataset of valid / high-quality designs and care about manufacturability.

L2 – Strategy C: direct conditional generation

Train a conditional generator $G_\theta(\mathrm{z}, \mathrm{y}^\star)$ that outputs designs meeting $\mathrm{y}^\star$.

- Inference: sample z, pass it with $\mathrm{y}^\star$, get designs directly.
- Produces **multiple** candidates for the same target by varying z.
- No optimization loop at inference time.

Trade-off

Fastest at inference but most demanding at training: needs many (design, performance) pairs to cover the performance space.

L2 – Non-uniqueness and ill-posedness

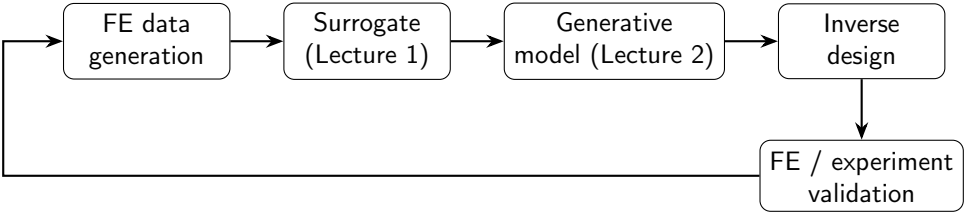
Multiple designs may match the same target. Options:

- **Embrace it.** Generate a *set* of diverse candidates; let the engineer pick.
- **Regularize.** Favor simpler, symmetric, manufacturable designs.
- **Multi-objective formulation.** For example, optimize performance *and* mass *and* robustness.

Honest expectation

Inverse design rarely gives a single answer. The win is **speed of exploration**, not a unique correct design.

L2 – The full data-driven design loop



- Every arrow can fail; validation closes the loop.
- The Day 3 workshops walk through pieces of this pipeline hands-on.

L2 – Open challenges

- **Data scarcity.** Simulations are expensive; experiments more so.
- **Validation & trust.** When do we believe a DL-generated design?
- **Manufacturability.** Bridging the gap from pretty voxels to printable parts.
- **Multi-physics, multi-scale.** Coupled FSI, rate-dependent materials, lattices.
- **Uncertainty propagation.** From noisy data $\rightarrow$ robust designs.

Good news

These are open problems you can contribute to, starting with this week's challenge.

Questions?